Chapter 5
Data Acquisition Features: SQL Database

In this chapter, you will be guided through two projects that demonstrate how to work with SQL databases as a source of data for analysis. This will build on the foundational application built in the previous two chapters.

This chapter will focus on SQL extracts. Since enterprise SQL databases tend to be very private, we'll guide the reader through creating an SQLite database first. This database will be a stand-in for a private enterprise database. Once there's a database available, we will look at extracting data from the database.

This chapter's projects cover the following essential skills:

- Building SQL databases.
- Extracting data from SQL databases.

The first project will build a SQL database for use by the second project.

In an enterprise environment, the source databases will already exist.

On our own personal computers, these databases don't exist. For this reason, we'll build a database in the first project, and extract from the database in the second project.

We'll start by looking at getting data into a SQL database. This will be a very small and simple database; the project will steer clear of the numerous sophisticated design complications for SQL data.

The second project will use SQL queries to extract data from the database. The objective is to produce data that is consistent with the projects in the previous chapters.

5.1 Project 1.4: A local SQL database

We'll often need data stored in a database that's accessed via the SQL query language. Use a search string like "SQL is the lingua franca" to find numerous articles offering more insight into the ubiquity of SQL. This seems to be one of the primary ways to acquire enterprise data for further analysis.

In the previous chapter, ***Chapter 4***, ***Data Acquisition Features: Web APIs and Scraping***, the projects acquired data from publicly available APIs and web pages. There aren't many publicly available SQL data sources. In many cases, there are dumps (or exports) of SQLite databases that can be used to build a local copy of the database. Direct access to a remote SQL database is not widely available. Rather than try to find access to a remote SQL database, it's simpler to create a local SQL database. The SQLite database is provided with Python as part of the standard library, making it an easy choice.

You may want to examine other databases and compare their features with SQLite. While some databases offer numerous capabilities, doing SQL extracts rarely seems to rely on anything more sophisticated than a basic `SELECT` statement. Using another database may require some changes to reflect that database's connections and SQL statement execution. For the most part, the DB-API interface in Python is widely used; there may be unique features for databases other than SQLite.

We'll start with a project to populate the database. Once a database is available, you can then move on to a more interesting project to extract the data using SQL statements.

5.1.1 Description

The first project for this chapter will prepare a SQL database with data to analyze. This is a necessary preparation step for readers working outside an enterprise environment with accessible SQL databases.

One of the most fun small data sets to work with is Anscombe's Quartet.

https://www.kaggle.com/datasets/carlmcbrideellis/data-anscombes-quartet

The URL given above presents a page with information about the CSV format file. Clicking the **Download** button will download the small file of data to your local computer.

The data is available in this book's GitHub repository's `data` folder, also.

In order to load a database, the first step is designing the database. We'll start with a look at some table definitions.

Database design

A SQL database is organized as tables of data. Each table has a fixed set of columns, defined as part of the overall database schema. A table can have an indefinite number of rows of data.

For more information on SQL databases, see **https://www.packtpub.com/product/learn-sql-database-programming/9781838984762** and **https://courses.packtpub.com/courses/sql**.

Anscombe's Quartet consists of four series of (*x*,*y*) pairs. In one commonly used source file, three of the series share common *x* values, whereas the fourth series has distinct *x* values.

A relational database often decomposes complicated entities into a collection of simpler entities. The objective is to minimize the repetitions of association types. The Anscombe's Quartet information has four distinct series of data values, which can be represented as the following two types of entities:

- The series is composed of a number of individual values. A table named `series_value` can store the individual values that are part of a series.
- A separate entity has identifying information for the series as a whole. A table named `sources` can store identifying information.

This design requires the introduction of key values to uniquely identify the series, and connect each value of a series with the summary information for the series.

For Anscombe's Quartet data, the summary information for a series is little more than a name.

This design pattern of an overall summary and supporting details is so common that it is essential for this project to reflect that common pattern.

See ***Figure 5.1*** for an ERD that shows the two tables that implement these entities and their relationships.

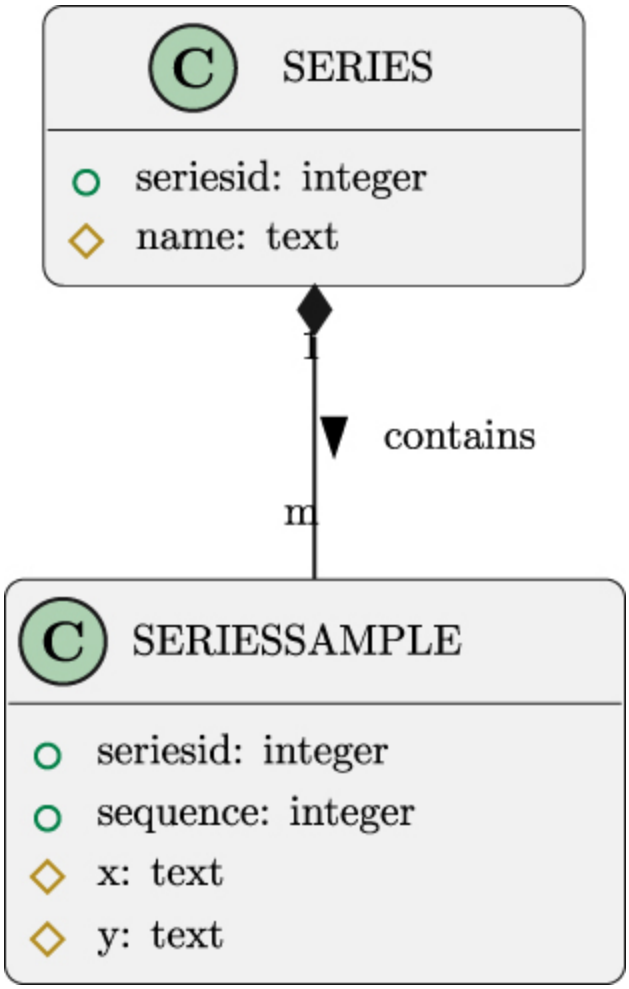

Figure 5.1: The Database Schema

This project will create a small application to build this schema of two tables. This application will can then load data into these tables.

Data loading

The process of loading data involves three separate operations:

- Reading the source data from a CSV (or other format) file.
- Executing SQL `INSERT` statements to create rows in tables.
- Executing a `COMMIT` to finalize the transaction and write data to the underlying database files.

Prior to any of these steps, the schema must be defined using `CREATE TABLE` statements.

In a practical application, it's also common to offer a composite operation to drop the tables, recreate the schema, and then load the data. The rebuilding often happens when exploring or experimenting with database designs. Many times, an initial design will prove unsatisfactory, and changes are needed. Additionally, the idea of building (and rebuilding) a small database will also be part of the acceptance test for any data acquisition application.

In the next section, we'll look at how to create a SQL database that can serve as a surrogate for a production database in a large enterprise.

5.1.2 Approach

There are two general approaches to working with SQL databases for this kind of test or demonstration application:

- Create a small application to build and populate a database.
- Create a SQL script via text formatting and run this through the database's CLI application. See **https://sqlite.org/cli.html**.

The small application will make use of the database client connection to execute SQL statements. In this case, a single, generic `INSERT` statement template with placeholders can be used. The client connection can provide values for the placeholders. While the application isn't complex, it will require unit and acceptance test cases.

The SQL script alternative uses a small application to transform data rows into valid `INSERT` statements. In many cases, a text editor search-and-replace can transform data text into `INSERT` statements. For more complex cases, Python f-strings can be used. The f-string might look like the following:

```
print(
    f"INSERT INTO SSAMPLES(SERIES, SEQUENCE, X, Y)"
```

```
        f"VALUES({series}, {sequence}, '{x}', '{y}')"
    )
```

This is often successful but suffers from a potentially severe problem: a *SQL injection exploit*.

The SQL injection exploit works by including an end-of-string-literal apostrophe `'` in a data value. This can lead to an invalid SQL statement. In extreme cases, it can allow injecting additional SQL statements to transform the `INSERT` statement into a script. For more information, see **https://owasp.org/www-community/attacks/SQL_Injection**. Also, see **https://xkcd.com/327/** for another example of a SQL injection exploit.

While SQL injection can be used maliciously, it can also be a distressingly common accident. If a text data value happens to have `'` in it, then this can create a statement in the SQL script file that has invalid syntax. SQL cleansing only defers the problem to the potentially complicated SQL cleansing function.

It's simpler to avoid building SQL text in the first place. A small application can be free from the complications of building SQL text.

We'll start by looking at the data definition for this small schema. Then we'll look at the data manipulation statements. This will set the stage for designing the small application to build the schema and load the data.

SQL Data Definitions

The essential data definition in SQL is a table with a number of columns (also called *attributes*). This is defined by a `CREATE TABLE` statement. The list of columns is provided in this statement. In addition to the columns, the language permits table constraints to further refine how a table may be used. For our purposes, the two tables can be defined as follows:

-

```
CREATE TABLE IF NOT EXISTS series(
  series_id INTEGER,
  name TEXT,

  PRIMARY KEY (series_id)
);

CREATE TABLE IF NOT EXISTS series_sample(
  series_id INTEGER,
  sequence INTEGER,
  x TEXT,
  y TEXT,

  PRIMARY KEY (series_id, sequence),
  FOREIGN KEY (series_id) REFERENCES series(series_id)
);
```

To remove a schema, the `DROP TABLE IF EXISTS series_sample` and `DROP TABLE IF EXISTS series` statements will do what's needed. Because of the foreign key reference, some databases make it necessary to remove all of the related `series_sample` rows before a `series` row can be removed.

The `IF EXISTS` and `IF NOT EXISTS` clauses are handy when debugging. We may, for example, change the SQL and introduce a syntax error into one of the `CREATE TABLE` statements. This can leave an incomplete schema. After fixing the problem, simply rerunning the entire sequence of `CREATE TABLE` statements will create only the tables that were missing.

An essential feature of this example SQL data model is a simplification of the data types involved. Two columns of data in the `series_sample` table are both defined as `TEXT`. This is a rarity; most SQL databases will use one of the available numeric types.

While SQL data has a variety of useful types, the raw data from other applications, however, isn't numeric. CSV files and HTML pages only provide text. For this reason, the results from this application need to be text, also. Once the tables are defined, an application can insert rows.

SQL Data Manipulations

New rows are created with the `INSERT` statement. While SQLite allows some details to be omitted, we'll stick with a slightly wordier but more explicit statement. Rows are created in the two tables as follows:

```
INSERT INTO series(series_id, name) VALUES(:series_id, :name)

INSERT INTO series_sample(series_id, sequence, x, y)
  VALUES(:series_id, :sequence, :x, :y)
```

The identifiers with a colon prefix, `:x`, `:y`, `:series_id`, etc., are parameters that will be replaced when the statement is executed. Since these replacements don't rely on SQL text rules — like the use of apostrophes to end a string — any value can be used.

It's rare to need to delete rows from these tables. It's easier (and sometimes faster) to drop and recreate the tables when replacing the data.

SQL Execution

Python's SQLite interface is the `sqlite3` module. This conforms to the PEP-249 standard (**https://peps.python.org/pep-0249/**) for database access. An application will create a database connection in general. It will use the connection to create a *cursor*, which can query or update the database.

The connection is made with a connection string. For many databases, the connection string will include the server hosting the database, and the database name; it may also include security credentials or other

options. For SQLite, the connection string can be a complete URI with the form `file:filename.db`. This has a scheme, `file:` and a path to the database file.

It's not required by this application, but a common practice is to sequester the SQL statements into a configuration file. Using a TOML format can be a handy way to separate the processing from the SQL statements that implement the processing. This separation permits small SQL changes without having to change the source files. For compiled languages, this is essential. For Python, it's a helpful way to make SQL easier to find when making database changes.

A function to create the schema might look like this:

```
CREATE_SERIES = """
CREATE TABLE IF NOT EXISTS series(
-- rest of the SQL shown above...
"""

CREATE_VALUES = """
CREATE TABLE IF NOT EXISTS series_sample(
-- rest of the SQL shown above...
"""

CREATE_SCHEMA = [
    CREATE_SERIES,
    CREATE_VALUES
]

def execute_statements(
        connection: sqlite3.Connection,
        statements: list[str]
) -> None:
    for statement in statements:
        connection.execute(statement)
    connection.commit()
```

The `CREATE_SCHEMA` is the sequence of statements required to build the schema. A similar sequence of statements can be defined to drop the schema. The two sequences can be combined to drop and recreate the schema as part of ordinary database design and experimentation.

A main program can create the database with code similar to the following:

```
with sqlite3.connect("file:example.db", uri=True) as connection:
    schema_build_load(connection, config, data_path)
```

This requires a function, `schema_build_load()`, to drop and recreate the schema and then load the individual rows of data.

We'll turn to the next step, loading the data. This begins with loading the series definitions, then follows this with populating the data values for each series.

Loading the SERIES table

The values in the `SERIES` table are essentially fixed. There are four rows to define the four series.

Executing a SQL data manipulation statement requires two things: the statement and a dictionary of values for the placeholders in the statement.

In the following code sample, we'll define the statement, as well as four dictionaries with values for placeholders:

```
INSERT_SERIES = """
    INSERT INTO series(series_id, name)
        VALUES(:series_id, :name)
"""

SERIES_ROWS = [
    {"series_id": 1, "name": "Series I"},
```

```
        {"series_id": 2, "name": "Series II"},
        {"series_id": 3, "name": "Series III"},
        {"series_id": 4, "name": "Series IV"},
    ]

    def load_series(connection: sqlite3.Connection) -> None:
        for series in SERIES_ROWS:
            connection.execute(INSERT_SERIES, series)
        connection.commit()
```

The `execute()` method of a connection object is given the SQL statement with placeholders and a dictionary of values to use for the placeholders. The SQL template and the values are provided to the database to insert rows into the table.

For the individual data values, however, something more is required. In the next section, we'll look at a transformation from source CSV data into a dictionary of parameter values for a SQL statement.

Loading the SERIES_VALUE table

It can help to refer back to the project in ***Chapter 3***, ***Project 1.1: Data Acquisition Base Application***. In this chapter, we defined a dataclass for the (*x*,*y*) pairs, and called it `XYPair`. We also defined a class hierarchy of `PairBuilder` to create `XYPair` objects from the CSV row objects.

It can be confusing to load data using application software that is suspiciously similar to the software for extracting data.

This confusion often arises in cases like this where we're forced to build a demonstration database.

It can also arise in cases where a test database is needed for complex analytic applications.

In most enterprise environments, the databases already exist and are already full of data. Test databases are still needed to confirm that analytic applications work.

The `INSERT` statement, shown above in ***SQL Data Manipulations*** has four placeholders. This means a dictionary with four parameters is required by the `execute()` method of a connection.

The `dataclasses` module includes a function, `asdict()`, to transform the object of the `XYPair` into a dictionary. This has two of the parameters required, `:x` and `:y`.

We can use the `|` operator to merge two dictionaries together. One dictionary has the essential attributes of the object, created by `asdict()`. The other dictionary is the SQL overheads, including a value for `:series_id`, and a value for `:sequence`.

Here's a fragment of code that shows how this might work:

```
    for sequence, row in enumerate(reader):
        for series_id, extractor in SERIES_BUILDERS:
            param_values = (
                asdict(extractor(row)) |
                {"series_id": series_id, "sequence": sequence}
            )
            connection.execute(insert_values_SQL, param_values)
```

The `reader` object is a `csv.DictReader` for the source CSV data. The `SERIES_BUILDERS` object is a sequence of two-tuples with the series number and a function (or callable object) to extract the appropriate columns and build an instance of `XYPair`.

For completeness, here's the value of the `SERIES_BUILDERS` object:

```
        SERIES_BUILDERS = [
        (1, series_1),
        (2, series_2),
        (3, series_3),
```

```
      (4, series_4)
  ]
```

In this case, individual functions have been defined to extract the required columns from the CSV source dictionary and build an instance of `XYPair`.

The above code snippets need to be built as proper functions and used by an overall `main()` function to drop the schema, build the schema, insert the values for the `SERIES` table, and then insert the `SERIES_VALUE` rows.

A helpful final step is a query to confirm the data was loaded. Consider something like this:

```
SELECT s.name, COUNT(*)
  FROM series s JOIN series_sample sv
    ON s.series_id = sv.series_id
  GROUP BY s.series_id
```

This should report the names of the four series and the presence of 11 rows of data.

5.1.3 Deliverables

There are two deliverables for this mini-project:

- A database for use in the next project. The primary goal is to create a database that is a surrogate for a production database in use by an enterprise.
- An application that can build (and rebuild) this database. This secondary goal is the means to achieve the primary goal.

Additionally, of course, unit tests are strongly encouraged. This works out well when the application is designed for testability. This means two features are essential:

- The database connection object is created in the `main()` function.
- The connection object is passed as an argument value to all the other functions that interact with the database.

Providing the connection as a parameter value makes it possible to test the various functions isolated from the overhead of a database connection. The tests for each application function that interacts with the database are given a mock connection object. Most mock connection objects have a mock `execute()` method, which returns a mock cursor with no rows. For queries, the mock `execute()` method can return mocked data rows, often something as simple as a `sentinel` object.

After exercising a function, the mock `execute()` method can then be examined to be sure the statement and parameters were provided to the database by the application.

A formal acceptance test for this kind of one-use-only application seems excessive. It seems easier to run the application and look at the results with a SQL `SELECT` query. Since the application drops and recreates the schema, it can be re-run until the results are acceptable.

5.2 Project 1.5: Acquire data from a SQL extract

At this point, you now have a useful SQL database with schema and data. The next step is to write applications to extract data from this database into a useful format.

5.2.1 Description

It can be difficult to use an operational database for analytic processing. During normal operations, locking is used to assure that database changes don't conflict with or overwrite each other. This locking can interfere with gathering data from the database for analytic purposes.

There are a number of strategies for extracting data from an operational database. One technique is to make a backup of the operational database and restore it into a temporary clone database for analytic purposes. Another technique is to use any replication features and do analytical work in the replicated database.

The strategy we'll pursue here is the "table-scan" approach. It's often possible to do rapid queries without taking out any database locks. The data may be inconsistent because of in-process transactions taking place at the time the query was running. In most cases, the number of inconsistent entities is a tiny fraction of the available data.

If it's necessary to have a *complete and consistent* snapshot at a specific point in time, the applications need to have been designed with this idea in mind. It can be very difficult to establish the state of a busy database with updates being performed by poorly designed applications. In some cases, the definitions of *complete* and *consistent* may be difficult to articulate because the domain of state changes isn't known in enough detail.

It can be frustrating to work with poorly designed databases.

It's often important to educate potential users of analytic software on the complexities of acquiring the data. This education needs to translate the database complications into the effect on the decisions they're trying to make and the data that supports those decisions.

The **User Experience** (**UX**) will be a command-line application. Our expected command line should look something like the following:

```
% python src/acquire.py -o quartet --schema extract.toml \
  --db_uri file:example.db -u username

Enter your password:
```

The `-o quartet` argument specifies a directory into which four results are written. These will have names like `quartet/series_1.json`.

The `--schema extract.toml` argument is the name of a file with the SQL statements that form the basis for the database queries. These are kept separate from the application to make it slightly easier to respond to the database structure changes without rewriting the application program.

The `--db_uri file:example.db` argument provides the URI for the database. For SQLite, the URIs have a scheme of `file:` and a path to the database file. For other database engines, the URI may be more complicated.

The `-u` argument provides a username for connecting to the database. The password is requested by an interactive prompt. This keeps the password hidden.

The UX shown above includes a username and password.

While it won't actually be needed for SQLite, it will be needed for other databases.

5.2.2 The Object-Relational Mapping (ORM) problem

A relational database design decomposes complicated data structures into a number of simpler entity types, which are represented as tables. The process of decomposing a data structure into entities is called *normalization*. Many database designs fit a pattern called *Third Normal Form*; but there are additional normalization forms. Additionally, there are compelling reasons to break some of the normalization rules to improve performance.

The relational normalization leads to a consistent representation of data via simple tables and columns. Each column will have an atomic value that cannot be further decomposed. Data of arbitrary complexity can be represented in related collections of flat, normalized tables.

See **https://www.packtpub.com/product/basic-relational-database-design-video/9781838557201** for some more insights into the database design activity.

The process of retrieving a complex structure is done via a relational *join* operation. Rows from different tables and joined into a result set from which Plain Old Python Objects can be constructed. This join operation is part of the `SELECT` statement. It appears in the `FROM` clause as a rule that states how to match rows in one table with rows from another table.

This distinction between relational design and object-oriented design is sometimes called the *Object-Relational Impedance Mismatch*. For more background, see **https://wiki.c2.com/?ObjectRelationalImpedanceMismatch**.

One general approach to reading complex data from a relational database is to create to an ORM layer. This layer uses SQL SELECT statements to extract data from multiple tables to build a useful object instance. The ORM layer may use a separate package, or it may be part of the application. While an ORM design can be designed poorly — i.e. the ORM-related operations may be scattered around haphazardly — the layer is always present in any application.

There are many packages in the **Python Package Index**(**PyPI**) that offer elegant, generalized ORM solutions. The **SQLAlchemy** (**https://www.sqlalchemy.org**) package is very popular. This provides a comprehensive approach to the entire suite of **Create, Retrieve, Update, and Delete**(**CRUD**) operations.

There are two conditions that suggest creating the ORM layer manually:

- Read-only access to a database. A full ORM will include features for operations that won't be used.
- An oddly designed schema. It can sometimes be difficult to work out an ORM definition for an existing schema with a design that doesn't fit the ORM's built-in assumptions.

There's a fine line between a bad database design and a confusing database design. A bad design has quirky features that cannot be successfully described through an ORM layer. A confusing design can be described, but it may require using "advanced" features of the ORM package. In many cases, building the ORM mapping requires learning enough about the ORM's capabilities to see the difference between bad and confusing.

In many cases, a relational schema may involve a vast number of interrelated tables, sometimes from a wide variety of subject areas. For example, there may be products and a product catalog, sales records for products, and inventory information about products. What is the proper boundary for a "product" class? Should it include everything in the database related to a product? Or should it be limited by some bounded context or problem domain?

Considerations of existing databases should lead to extensive conversations with users on the problem domain and context. It also leads to further conversations with the owners of the applications creating the data. All of the conversations are aimed at understanding how a user's concept may overlap with existing data sources.

Acquiring data from relational databases can be a challenge.

The relational normalization will lead to complications. The presence of overlapping contexts can lead to further complications.

What seems to be helpful is providing a clear translation from the technical world of the database to the kinds of information and decisions users want to make.

5.2.3 About the source data

See ***Figure 5.2*** for an ERD that shows the two tables that provide the desired entities:

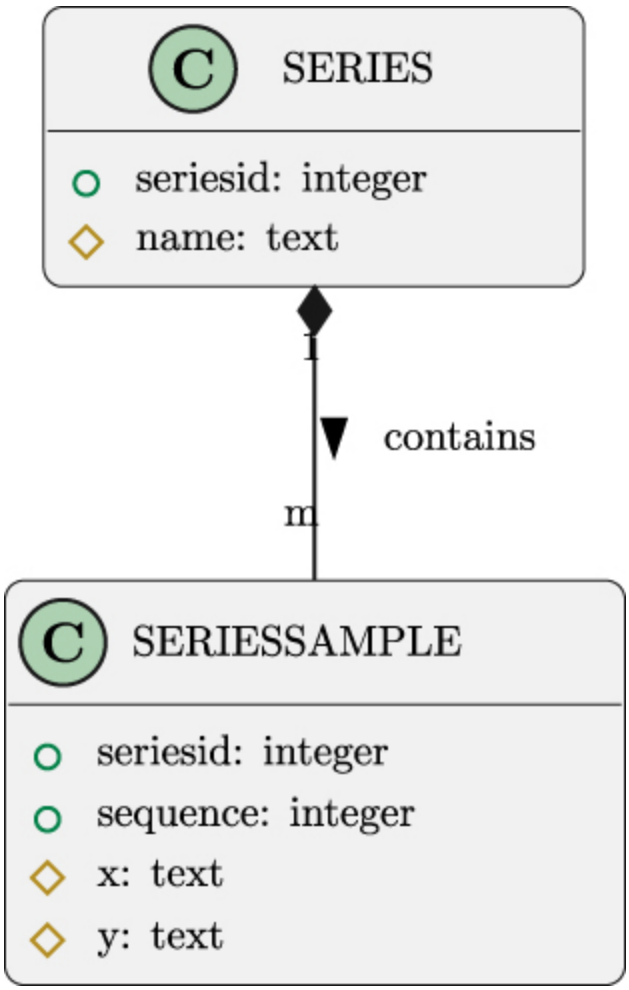

Figure 5.2: The Database Schema

In the design shown above, two tables decompose instances of the `Series` class. Here are the Python class definitions:

```
from dataclasses import dataclass

@dataclass
```

```
class SeriesSample:
    x: str
    y: str

@dataclass
class Series:
    name: str
    samples: list[SeriesSample]
```

The idea here is that a collection of `SeriesSample` objects are part of a single composite `Series` object. The `SeriesSample` objects, separated from the containing `Series`, aren't useful in isolation. A number of `SeriesSample` instances depend on a `Series` object.

There are three general approaches to retrieving information from a normalized collection of tables:

- A single SQL query. This forces the database server to **join** rows from multiple tables, providing a single result set.
- A series of queries to extract data from separate tables and then do lookups using Python dictionaries.
- Nested SQL queries. These use simpler SQL but can make for a large number of database requests.

Neither alternative is a perfect solution in all cases. Many database designers will insist that database join operations are magically the fastest. Some actual timing information suggests that Python dictionary lookups can be much faster. Numerous factors impact query performance and the prudent design is to implement alternatives and compare performance.

The number of factors influencing performance is large. No simple "best practice" exists. Only actual measurements can help to make a design decision.

The join query to retrieve the data might look this:

```
  SELECT s.name, sv.x, sv.y
    FROM series s JOIN series_sample sv ON s.series_id = sv.series_id
```

Each distinct value of `s.name` will lead to the creation of a distinct `Series` object. Each row of `sv.x`, and `sv.y` values becomes a `SeriesSample` instance within the `Series` object.

Building objects with two separate `SELECT` statements involves two simpler queries. Here's the "outer loop" query to get the individual series:

```
  SELECT s.name, s.series_id
    FROM series s
```

Here's the "inner loop" query to get rows from a specific series:

```
  SELECT sv.x, sv.y
    FROM series_sample sv
    WHERE sv.series_id = :series_id
    ORDER BY sv.sequence
```

The second `SELECT` statement has a placeholder that depends on the results of the first query. The application must provide this parameter when making a nested request for a series-specific subset of rows from the `series_sample` table.

It's also important to note the output is expected to be pure text, which will be saved in ND JSON files. This means the sophisticated structure of the SQL database will be erased.

This will also make the interim results consistent with CSV files and HTML pages, where the data is only text. The output should be similar to the output from the CSV extract in ***Chapter 3***, ***Project 1.1: Data Acquisition Base Application***: a file of small JSON documents that have the keys `"x"` and `"y"`. The goal is to strip away structure that may have been imposed by the data persistence mechanism — a SQL

database for this project. The data is reduced into a common base of text.

In the next section, we'll look more closely at the technical approach to acquiring data from a SQL database.

5.2.4 Approach

We'll take some guidance from the C4 model (**https://c4model.com**) when looking at our approach.

- **Context**: For this project, a context diagram would show a user extracting data from a source. The reader may find it helpful to draw this diagram.
- **Containers**: One container is the user's personal computer. The other container is the database server, which is running on the same computer.
- **Components**: We'll address the components below.
- **Code**: We'll touch on this to provide some suggested directions.

This project adds a new `db_client` module to extract the data from a database. The overall application in the `acquire` module will change to make use of this new module. The other modules — for the most part — will remain unchanged.

The component diagram in ***Figure 5.3*** shows an approach to this project.

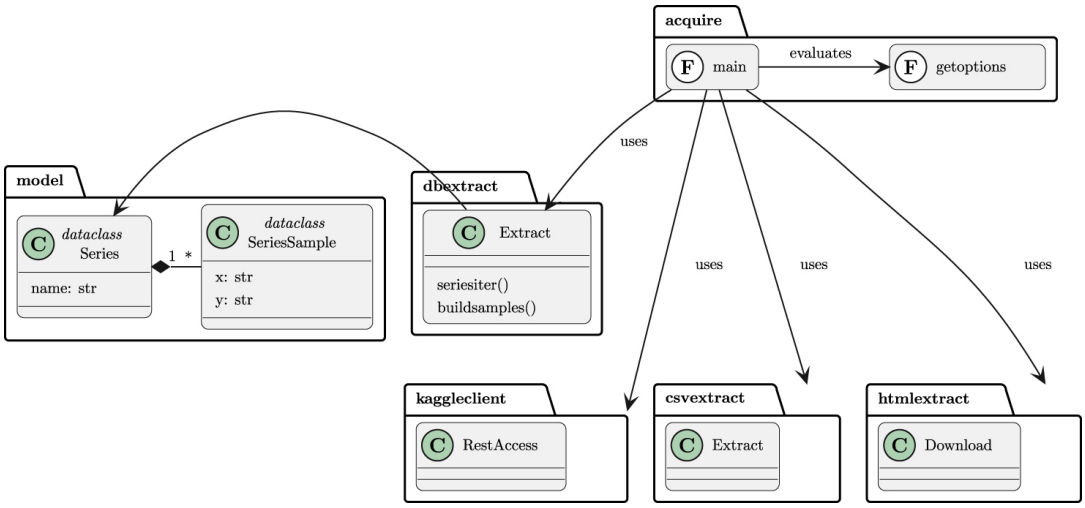

Figure 5.3: Component Diagram

This diagram shows a revision to the underlying `model`. This diagram extends the `model` module to make the distinction between the composite series object and the individual samples within the overall series. It also renames the old `XYPair` class to a more informative `SeriesSample` class.

This distinction between series has been an implicit part of the project in the previous chapters. At this point, it seems potentially helpful to distinguish a collection of samples from an individual sample.

Some readers may object to renaming a class partway through a series of closely related projects. This kind of change is — in the author's experience — very common. We start with an understanding that evolves and grows the more we work the problem domain, the users, and the technology. It's very difficult to pick a great name for a concept. It's more prudent to fix names as we learn.

The new module will make use of two SQL queries to perform the extract. We'll look at these nested requests in the next section.

Extract from a SQL DB

The extraction from the database constructs a series of two parts. The first part is to get the attributes of the `Series` class. The second part is

to get each of the individual `SeriesSample` instances.

Here's the overview of a potential class design:

```
import model
import sqlite3
from typing import Any
from collections.abc import Iterator

class Extract:
    def build_samples(
            self,
            connection: sqlite3.Connection,
            config: dict[str, Any],
            name: str
    ) -> model.Series:
        ...

    def series_iter(
            self,
            connection: sqlite3.Connection,
            config: dict[str, Any]
    ) -> Iterator[model.Series]:
        ...
```

The `series_iter()` method iterates over the `Series` instances that can be created from the database. The `build_samples()` method creates the individual samples that belong to a series.

Here's a first draft of an implementation of the `build_samples()` method:

```
def build_samples(
        self,
        connection: sqlite3.Connection,
        config: dict[str, Any],
        name: str
) -> list[model.SeriesSample]:
    samples_cursor = connection.execute(
```

```
            config['query']['samples'],
            {"name": name}
        )
        samples = [
            model.SeriesSample(
                x=row[0],
                y=row[1])
            for row in samples_cursor
        ]
        return samples
```

This method will extract the collection of samples for a series given the name. It relies on the SQL query in the `config` object. The list of samples is built from the results of the query using a list comprehension.

This first draft implementation has a dependency on the `SeriesSample` class name. This is another SOLID design issue, similar to the one in ***Class design*** of ***Chapter 3***, ***Project 1.1: Data Acquisition Base Application***.

A better implementation would replace this direct dependency with a dependency that can be injected at runtime, permitting better isolation for unit testing.

Here's an implementation of the `series_iter()` method:

```
def series_iter(
        self,
        connection: sqlite3.Connection,
        config: dict[str, Any]
) -> Iterator[model.Series]:
    print(config['query']['names'])
    names_cursor = connection.execute(config['query']['names'])
    for row in names_cursor:
        name=row[0]
        yield model.Series(
            name=name,
```

```
            samples=self.build_samples(connection, config, name)
        )
```

This method will extract each of the series from the database. It, too, gets the SQL statements from a configuration object, `config`. A configuration object is a dictionary of dictionaries. This structure is common for TOML files.

The idea is to have a configuration file in TOML notation that looks like this:

```
[query]
summary =  """
SELECT s.name, COUNT(*)
  FROM series s JOIN series_sample sv ON s.series_id = sv.series_id
  GROUP BY s.series_id
"""

detail =  """
SELECT s.name, s.series_id, sv.sequence, sv.x, sv.y
  FROM series s JOIN series_value sv ON s.series_id = sv.series_id
"""

names = """
SELECT s.name FROM series s
"""

samples = """
SELECT sv.x, sv.y
  FROM series_sample sv JOIN series s ON s.series_id = sv.series_id
  WHERE s.name = :name
  ORDER BY sv.sequence
"""
```

This configuration has a `[query]` section, with several individual SQL statements used to query the database. Because the SQL statements are often quite large, triple quotes are used to delimit them.

In cases where the SQL statements are very large, it's can seem helpful to put them in separate files. This leads to a more complicated configuration with a number of files, each with a separate SQL statement.

Before we look at the deliverables, we'll talk a bit about why this data acquisition application is different from the previous projects.

SQL-related processing distinct from CSV processing

It's helpful to note some important distinctions between working with CSV data and working with SQL data.

First, CSV data is always text. When working with a SQL database, the underlying data often has a data type that maps pleasantly to a native Python type. SQL databases often have a few numeric types, including integers and floating-point numbers. Some databases will handle decimal values that map to Python's `decimal.Decimal` class; this isn't a universal capability, and some databases force the application to convert between `decimal.Decimal` and text to avoid the truncation problems inherent with floating-point values.

The second important distinction is the tempo of change. A SQL database schema tends to change slowly, and change often involves a review of the impact of the change. In some cases, CSV files are built by interactive spreadsheet software, and manual operations are used to create and save the data. Unsurprisingly, the interactive use of spreadsheets leads to small changes and inconsistencies over short periods of time. While some CSV files are produced by highly automated tools, there may be less scrutiny applied to the order or names of columns.

A third important distinction relates to the design of spreadsheets contrasted with the design of a database. A relational database is often highly normalized; this is an attempt to avoid redundancy. Rather than repeat a group of related values, an entity is assigned to a separate table with a primary key. References to the group of values via the

primary key are used to avoid repetition of the values themselves. It's less common to apply normalization rules to a spreadsheet.

Because spreadsheet data may not be fully normalized, extracting meaningful data from a spreadsheet often becomes a rather complicated problem. This can be exacerbated when spreadsheets are tweaked manually or the design of the spreadsheet changes suddenly. To reflect this, the designs in this book suggest using a hierarchy of classes — or collection of related functions — to build a useful Python object from a spreadsheet row. It is often necessary to keep a large pool of builders available to handle variant spreadsheet data as part of historical analysis.

The designs shown earlier had a `PairBuilder` subclass to create individual sample objects. These designs used an `Extract` class to manage the overall construction of samples from the source file. This provided flexibility to handle spreadsheet data.

A database extract is somewhat less likely to need a flexible hierarchy of objects to create useful Python objects. Instead, the needed flexibility is often implemented by changing SQL statements to reflect schema changes or a deeper understanding of the available data. For this reason, we encourage the use of a TOML-format file to keep the SQL statements, permitting some changes without having to add more subclasses to the Python application. The TOML-format configuration files can have version numbers in the file name (and in the comments) to make it clear which database schema they are designed against.

Now that we have a design approach, it's important to make sure we have a list of deliverables that serve as a definition of "Done."

5.2.5 Deliverables

This project has the following deliverables:

- Documentation in the `docs` folder.
- Acceptance tests in the `tests/features` and `tests/steps` folders.
- The acceptance tests will involve creating and destroying example databases as test fixtures.
- Unit tests for application modules in the `tests` folder.
- Mock objects for the database connection will be part of the unit tests.
- Application to acquire data from a SQL database.

We'll look at a few of these deliverables in a little more detail.

Mock database connection and cursor objects for testing

For the data acquisition application, it's essential to provide a mock connection object to expose the SQL and the parameters that are being provided to the database. This mock object can also provide a mock cursor as a query result.

As noted earlier in ***Deliverables***, this means the connection object should be created only in the `main()` function. It also means the connection object should be a parameter to any other functions or methods that perform database operations. If the connection object is referenced consistently, it becomes easier to test by providing a mock connection object.

We'll look at this in two parts: first, the conceptual Given and When steps; after that, we'll look at the Then steps. This is sometimes called "arrange-act-assert". Here's the start of the **PyTest** test case:

```
import sqlite3
from typing import Any, cast
from unittest.mock import Mock, call, sentinel
from pytest import fixture
import db_extract
import model
```

```
def test_build_sample(
        mock_connection: sqlite3.Connection,
        mock_config: dict[str, Any]
):
    extract = db_extract.Extract()
    results = list(
        extract.series_iter(mock_connection, mock_config)
    )
```

The assertions confirm the results come from the mock objects without being transformed, dropped, or corrupted by some error in the code under test. The assertions look like this example:

```
    assert results == [
        model.Series(
            name=sentinel.Name,
            samples=[
               model.SeriesSample(sentinel.X, sentinel.Y)
            ]
        )
    ]
    assert cast(Mock, mock_connection).execute.mock_calls == [
        call(sentinel.Names_Query),
        call(sentinel.Samples_Query, {'name': sentinel.Name})
    ]
```

A mock connection object must provide results with sentinel objects that have the proper structure to look like the iterable `Cursor` object that is returned by SQLite3 when executing a database query.

The mock connection seems rather complicated because it involves two separate mock cursors and a mock connection. Here's some typical code for a mock connection:

```
@fixture
def mock_connection() -> sqlite3.Connection:
    names_cursor: list[tuple[Any, ...]] = [
        (sentinel.Name,)
    ]
```

```
    samples_cursor: list[tuple[Any, ...]]  = [
        (sentinel.X, sentinel.Y)
    ]
    query_to_cursor: dict[sentinel, list[tuple[Any, ...]]] = {
        sentinel.Names_Query: names_cursor,
        sentinel.Samples_Query: samples_cursor
    }

    connection = Mock(
        execute=Mock(
            side_effect=lambda query, param=None: query_to_cursor[query]
        )
    )
    return cast(sqlite3.Connection, connection)
```

The mocked cursors are provided as simple lists. If the code under test used other features of a cursor, a more elaborate `Mock` object would be required. The `query_to_cursor` mapping associates a result with a particular query. The idea here is the queries will be `sentinel` objects, not long SQL strings.

The `connection` object uses the side-effect feature of `Mock` objects. When the `execute()` method is evaluated, the call is recorded, and the result comes from the side-effect function. In this case, it's a lambda object that uses the `query_to_cursor` mapping to locate an appropriate cursor result.

This use of the side-effect feature avoids making too many assumptions about the internal workings of the unit under test. The SQL will be a `sentinel` object and the results will contain `sentinel` objects.

In this case, we're insisting the unit under test does no additional processing on the values retrieved from the database. In other applications, where additional processing is being done, more sophisticated mock objects or test literals may be required.

It's not unusual to use something like `(11, 13)` instead of `(sentinel.X, sentinel.Y)` to check that a computation is being

performed correctly. However, it's more desirable to isolate the computations performed on SQL results into separate functions. This allows testing these functions as separate units. The SQL retrieval processing can be tested using mock functions for these additional computations.

Also, note the use of the `cast()` function from the `typing` module to tell tools like **mypy** this object can be used like a `Connection` object.

Unit test for a new acquisition module

Throughout this sequence of chapters, the overall `acquisition` module has grown more flexible. The idea is to permit a wide variety of data sources for an analysis project.

Pragmatically, it is more likely to modify an application to work with a number of distinct CSV formats, or a number of distinct database schemas. When a RESTful API changes, it's often a good strategy to introduce new classes for the changed API as an alternatives to existing classes. Simply modifying or replacing the old definition — in a way — erases useful history on why and how an API is expected to work. This is the Open/Closed principle from the SOLID design principles: the design is open to extension but closed to modification.

Acquiring data from a wide variety of data sources — as shown in these projects — is less likely than variations in a single source. As an enterprise moves from spreadsheets to central databases and APIs, then the analytical tools should follow the data sources.

The need for flexible data acquisition drives the need to write unit tests for the acquisition module to both cover the expected cases and cover the potential domain of errors and mistakes in use.

Acceptance tests using a SQLite database

The acceptance tests need to create (and destroy) a test database. The tests often need to create, retrieve, update, and delete data in the test database to arrange data for the given step or assert the results in the Then step.

In the context of this book, we started with the ***Project 1.4: A local SQL database*** project to build a test database. There aren't many readily accessible, public, relational databases with extractable data. In most cases, these databases are wrapped with a RESTful API.

The database built in the previous project has two opposing use cases:

- It is for test purposes and can be deleted and rebuilt freely.
- This database must be treated as if it's precious enterprise data, and should not be deleted or updated.

When we think of the database created in ***Project 1.4: A local SQL database*** as if it were production data, we need to protect it from unexpected changes.

This means our acceptance tests must build a separate, small, test database, separate from the "production" database created by the previous project.

The test database must not collide with precious enterprise data.

There are two common strategies to avoid collisions between test databases and enterprise databases:

1. Use OS-level security in the file system to make it difficult to damage the files that comprise a shared database. Also, using strict naming conventions can put a test database into a separate namespace that won't collide with production databases.
2. Run the tests in a **Docker container** to create a virtual environment in which production data cannot be touched.

As we noted above in ***Approach***, the idea behind a database involves two containers:

- A container for the application components that extract the data.
- A container for the database components that provide the data. An acceptance test can create an ephemeral database service.

With SQLite, however, there is no distinct database service container. The database components become part of the application's components and run in the application's container. The lack of a separate service container means SQLite breaks the conceptual two-container model that applies to large, enterprise databases. We can't create a temporary, mock database **service** for testing purposes.

Because the SQLite database is nothing more than a file, we must focus on OS-level permissions, file-system paths, and naming conventions to keep our test database separate from the production database created in an earlier project. We emphasize this because working with a more complicated database engine (like MySQL or PostgreSQL) will also involve the same consideration of permissions, file paths, and naming conventions. Larger databases will add more considerations, but the foundations will be similar.

It's imperative to avoid disrupting production operations while creating data analytic applications.

Building and destroying a temporary SQLite database file suggests the use of a `@fixture` to create a database and populate the needed schema of tables, views, indexes, etc. The Given steps of individual scenarios can provide a summary of the data arrangement required by the test.

We'll look at how to define this as a feature. Then, we can look at the steps required for the implementation of the fixture, and the step definitions.

The feature file

Here's the kind of scenario that seems to capture the essence of a SQL extract application:

```
@fixture.sqlite
Scenario: Extract data from the enterprise database

  Given a series named "test1"
  And sample values "[(11, 13), (17, 19)]"
  When we run the database extract command with the test fixture database
  Then log has INFO line with "series: test1"
  And log has INFO line with "count: 2"
  And output directory has file named "quartet/test1.csv"
```

The `@fixture.` tag follows the common naming convention for associating specific, reusable fixtures with scenarios. There are many other purposes for tagging scenarios in addition to specifying the fixture to use. In this case, the fixture information is used to build an SQLite database with an empty schema.

The Given steps provide some data to load into the database. For this acceptance test, a single series with only a few samples is used.

The tag information can be used by the **behave** tool. We'll look at how to write a `before_tag()` function to create (and destroy) the temporary database for any scenario that needs it.

The sqlite fixture

The fixture is generally defined in the `environment.py` module that the **behave** tool uses. The `before_tag()` function is used to process the tags for a feature or a scenario within a feature. This function lets us then associate a specific feature function with the scenario:

```
from behave import fixture, use_fixture
from behave.runner import Context
```

```
def before_tag(context: Context, tag: str) -> None:
    if tag == "fixture.sqlite":
        use_fixture(sqlite_database, context)
```

The `use_fixture()` function tells the **behave** runner to invoke the given function, `sqlite_database()`, with a given argument value – in this case, the `context` object. The `sqlite_database()` function should be a generator: it can prepare the database, execute a `yield` statement, and then destroy the database. The **behave** runner will consume the yielded value as part of setting up the test, and the consume one more value when it's time to tear down the test.

The function to create (and destroy) the database has the following outline:

```
from collections.abc import Iterator
from pathlib import Path
import shutil
import sqlite3
from tempfile import mkdtemp
import tomllib

from behave import fixture, use_fixture
from behave.runner import Context

@fixture
def sqlite_database(context: Context) -> Iterator[str]:
    # Setup: Build the database files (shown later).

    yield context.db_uri

    # Teardown: Delete the database files (shown later).
```

We've decomposed this function into three parts: the setup, the `yield` to allow the test scenario to proceed, and the teardown. We'll look at the *Set up: Build the database files* and the *Teardown: Delete the database files* sections separately.

The setup processing of the `sqlite_database()` function is shown in the following snippet:

```
    # Get Config with SQL to build schema.
    config_path = Path.cwd() / "schema.toml"
    with config_path.open() as config_file:
        config = tomllib.load(config_file)
        create_sql = config['definition']['create']
        context.manipulation_sql = config['manipulation']
    # Build database file.
    context.working_path = Path(mkdtemp())
    context.db_path =  context.working_path / "test_example.db"
    context.db_uri = f"file:{context.db_path}"
    context.connection = sqlite3.connect(context.db_uri, uri=True)
    for stmt in create_sql:
        context.connection.execute(stmt)
    context.connection.commit()
```

The configuration file is read from the current working directory. The SQL statements to create the database and perform data manipulations are extracted from the schema. The database creation SQL will be executed during the tag discovery. The manipulation SQL will be put into the context for use by the Given steps executed later.

Additionally, the context is loaded up with a working path, which will be used for the database file as well as the output files. The context will have a `db_uri` string, which can be used by the data extract application to locate the test database.

Once the context has been filled, the individual SQL statements can be executed to build the empty database.

After the `yield` statement, the teardown processing of the `sqlite_database()` function is shown in the following snippet:

```
    context.connection.close()
    shutil.rmtree(context.working_path)
```

The SQLite3 database must be closed before the files can be removed. The `shutil` package includes functions that work at a higher level on files and directories. The `rmtree()` function removes the entire directory tree and all of the files within the tree.

This fixture creates a working database. We can now write step definitions that depend on this fixture.

The step definitions

We'll show two-step definitions to insert series and samples into the database. The following example shows the implementation of one of the `Given` steps:

```
@given(u'a series named "{name}"')
def step_impl(context, name):
    insert_series = context.manipulation_sql['insert_series']
    cursor = context.connection.execute(
        insert_series,
        {'series_id': 99, 'name': name}
    )
    context.connection.commit()
```

The step definition shown above uses SQL to create a new row in the `series` table. It uses the connection from the context; this was created by the `sqlite_database()` function that was made part of the testing sequence by the `before_tag()` function.

The following example shows the implementation of the other `Given` step:

```
@given(u'sample values "{list_of_pairs}"')
def step_impl(context, list_of_pairs):
    pairs = literal_eval(list_of_pairs)
    insert_values = context.manipulation_sql['insert_values']
    for seq, row in enumerate(pairs):
        cursor = context.connection.execute(
```

```
            insert_values,
            {'series_id': 99, 'sequence': seq, 'x': row[0], 'y': row[1]}
        )
    context.connection.commit()
```

The step definition shown above uses SQL to create a new row in the `series_sample` table. It uses the connection from the context, also.

Once the series and samples have been inserted into the database, the `When` step can run the data acquisition application using the database URI information from the context.

The `Then` steps can confirm the results from running the application match the database seeded by the fixture and the `Given` steps.

With this testing framework in place, you can run the acceptance test suite. It's common to run the acceptance tests before making any of the programming changes; this reveals the `acquire` application doesn't pass all of the tests.

In the next section, we'll look at the database extract module and re-write the main application.

The Database extract module, and refactoring

This project suggests three kinds of changes to the code written for the previous projects:

- Revise the `model` module to expand on what a "series" is: it's a parent object with a name and a list of subsidiary objects.
- Add the `db_extract` module to grab data from a SQL database.
- Update the `acquire` module to gather data from any of the available sources and create CSV files.

Refactoring the `model` module has a ripple effect on other projects, requiring changes to those modules to alter the data structure names.

As we noted in ***Approach***, it's common to start a project with an understanding that evolves and grows. More exposure to the problem domain, the users, and the technology shifts our understanding. This project reflects a shift in understanding and leads to a need to change the implementation of previously completed projects.

One consequence of this is exposing the series' name. In projects from previous chapters, the four series had names that were arbitrarily imposed by the application program. Perhaps a file name might have been `"series_1.csv"` or something similar.

Working with the SQL data exposed a new attribute, the name of a series. This leads to two profound choices for dealing with this new attribute:

1. Ignore the new attribute.
2. Alter the previous projects to introduce a series name.

Should the series name be the file name? This seems to be a bad idea because the series name may have spaces or other awkward punctuation.

It seems as though some additional metadata is required to preserve the series name and associate series names with file names. This would be an extra file, perhaps in JSON or TOML format, created as part of the extract operation.

5.3 Summary

This chapter's projects covered two following essential skills:

- Building SQL databases. This includes building a representative of a production database, as well as building a test database.
- Extracting data from SQL databases.

This requires learning some SQL, of course. SQL is sometimes called the *lingua franca* of data processing. Many organizations have SQL databases, and the data must be extracted for analysis.

Also important is learning to work in the presence of precious production data. It's important to consider the naming conventions, file system paths, and permissions associated with database servers and the files in use. Attempting to extract analytic data is not a good reason for colliding with production operations.

The effort required to write an acceptance test that uses an ephemeral database is an important additional skill. Being able to create databases for test purposes permits debugging by identifying problematic data, creating a test case around it, and then working in an isolated development environment. Further, having ephemeral databases permits examining changes to a production database that might facilitate analysis or resolve uncertainty in production data.

In the next chapter, we'll transition from the bulk acquisition of data to understanding the relative completeness and usefulness of the data. We'll build some tools to inspect the raw data that we've acquired.

5.4 Extras

Here are some ideas for the reader to add to this project.

5.4.1 Consider using another database

For example, MySQL or PostgreSQL are good choices. These can be downloaded and installed on a personal computer for non-commercial purposes. The administrative overheads are not overly burdensome.

It is essential to recognize these are rather large, complex tools. For readers new to SQL, there is a lot to learn when trying to install, con-

figure, and use one of these databases.

See **https://dev.mysql.com/doc/mysql-getting-started/en/** for some advice on installing and using MySQL.

See **https://www.postgresql.org/docs/current/tutorial-start.html** for advice on installing and using PostgreSQL.

In some cases, it makes sense to explore using a Docker container to run a database server on a virtual machine. See **https://www.packtpub.com/product/docker-for-developers/9781789536058** for more about using Docker as a way to run complex services in isolated environments.

See **https://dev.mysql.com/doc/refman/8.0/en/docker-mysql-getting-started.html** for ways to use MySQL in a Docker container.

See **https://www.docker.com/blog/how-to-use-the-postgres-docker-official-image/** for information on running PostgreSQL in a Docker container.

5.4.2 Consider using a NoSQL database

A NoSQL database offers many database features — including reliably persistent data and shared access — but avoids (or extends) the relational data model and replaces the SQL language.

This leads to data acquisition applications that are somewhat like the examples in this chapter. There's a connection to a server and requests to extract data from the server. The requests aren't SQL `SELECT` statements. Nor is the result necessarily rows of data in a completely normalized structure.

For example, MongoDB. Instead of rows and tables, the data structure is JSON documents. See

https://www.packtpub.com/product/mastering-mongodb-4x-second-edition/9781789617870.

The use of MongoDB changes data acquisition to a matter of locating the JSON documents and then building the desired document from the source data in the database.

This would lead to two projects, similar to the two described in this chapter, to populate the "production" Mongo database with some data to extract, and then writing the acquisition program to extract the data from the database.

Another alternative is to use the PostgreSQL database with JSON objects for the data column values. This provides a MongoDB-like capability using the PostgreSQL engine. See **https://www.postgresql.org/docs/9.3/functions-json.html** for more information on this approach.

Here are some common categories of NoSQL databases:

- Document databases
- Key-value stores
- Column-oriented databases
- Graph databases

The reader is encouraged to search for representative products in these categories and consider the two parts of this chapter: loading a database and acquiring data from the database.

5.4.3 Consider using SQLAlchemy to define an ORM layer

In ***The Object-Relational Mapping (ORM) problem*** we talked about the ORM problem. In that section, we made the case that using a tool

to configure an ORM package for an existing database can sometimes turn out badly.

This database, however, is very small. It's an ideal candidate for learning about simple ORM configuration.

We suggest starting with the SQLAlchemy ORM layer. See **https://docs.sqlalchemy.org/en/20/orm/quickstart.html** for advice on configuring class definitions that can be mapped to tables. This will eliminate the need to write SQL when doing extracts from the database.

There are other ORM packages available for Python, also. The reader should feel free to locate an ORM package and build the extraction project in this chapter using the ORM data model.